Policies Guiding CoE

Mugen Blue, Milin Doppalapudi, Aditya Petety

Abstract

The Chain-of-Experts (CoE) framework leverages Large Language Models (LLMs) to address complex real-world optimization challenges. In this system, multiple LLMs function as specialized experts, collaboratively transforming natural language problem descriptions into executable Python code. Our research focuses on developing alternative policies to replace the conductor LLM responsible for determining the decisions and sequence in which these experts are consulted. We evaluate the performance and associated costs of various approaches, including different LLM configurations, reinforcement learning (RL) strategies, and rule-based policies.

1 Introduction

Operations Research (OR) plays a crucial role in our daily lives, and in recent years it has gained significant prominence, with many universities establishing dedicated departments focused on OR. This field employs mathematical models to combat complex real-world optimization problems, many of which are commonly known. Some of the challenges addressed by the OR community include:

- Project Management (ex. resource allocation, task scheduling)
- Queuing Theory (ex. traffic flow, manufacturing bottlenecks)
- Strategic Analysis (ex. pricing strategy, performance analysis)

These problems have been extensively studied theoretically, aiming for optimal, approximate, and efficient solutions. However, a major drawback of theoretical approaches is that many algorithms can be overly complex or incur reduced performance in practical applications. This is where the OR community plays a crucial role, working to bridge the gap between theoretical research and real-world implementation.

Typically, these problems are not presented in a mathematical formulation to the expert attempting to solve them, but instead are commonly described in textual form. Although converting such textual descriptions into mathematical models is conceptually straightforward, it remains a time-intensive step for human experts. If a framework could automatically translate text descriptions into mathematical models—or even better, directly into executable code for a server or workstation—it would significantly reduce the time and effort required.

Large Language Models (LLMs) can act as this needed bridge between the user and the optimizer, as they can viably mimic the role of an expert dynamically. Currently, state-of-the-art (SoTA) Chain of Experts systems utilize specialized LLMs, each fine-tuned to perform a specific sub-task effectively. To manage the interaction between these specialized experts, an additional conductor LLM is employed to determine the optimal consultation order. Xiao et al. [2024]

However, issues arise due to large number of LLMs leading to be counterproductive with respect to efficiency and cost. Since most OR tasks require large models and it is non-ideal to replace the expert LLMs, the issue revolves around the conductor. Using a powerful conductor LLM to manage expert interactions often results in high token usage and increasing computational and financial

Preprint. Under review.

costs. The aim of this research is to explore simpler and more cost-effective policies as replacements for the conductor LLM. Specifically, we investigate alternative strategies such as rule-based policies and reinforcement learning (RL)-based approaches. By evaluating these simpler conductor policies against traditional LLM-based conductors, we seek to achieve significant reductions in token usage and computational costs without sacrificing performance in solving complex operational optimization problems.

2 Background

Integer Programming (IP) is a mathematical optimization technique where some or all decision variables are constrained to take integer values. This approach is particularly useful in scenarios where solutions must be whole numbers, such as cases of OR; scheduling, resource allocation, and network design. The standard integer programming (IP) used to solve most optimization problems is of this form: Ramamonjison et al [2022]

$$\begin{aligned} \text{Maximize} \quad & c^T x \\ \text{subject to} \quad & Ax \leq b, \\ & x \geq 0, \\ & x_i \in \mathbb{Z}, \quad \forall i \in \{1, \ldots, n\}. \end{aligned}$$

where:

- $x \in \mathbb{Z}^n$ is the vector of decision variables,
- $c \in \mathbb{R}^n$ is the coefficient vector in the objective function,
- $A \in \mathbb{R}^{m \times n}$ is the constraint coefficient matrix,
- $b \in \mathbb{R}^m$ is the right-hand side vector.

The objective function $c^T x$ represents the goal to be maximized, such as profit or efficiency. The constraints $Ax \leq b$ and $x \geq 0$ define the feasible region within which the solution must lie, ensuring all practical limitations and requirements are satisfied. The constraint $x_i \in \mathbb{Z}$ ensures that each decision variable takes on an integer value, which is essential in applications where fractional solutions are not meaningful.

Integer Programming is a fundamental tool in OR, enabling the modeling and solving of complex decision-making problems where discrete choices are involved.

3 Related Works

Chain of Experts

The CoE generates code from the natural language description of the optimization problem above Xiao et al. [2024]. The two main subtasks associated with this are

- Named entity recognition
- Generating the precise meaning representation

Chain or experts proposed the first novel multi-agent framework enabling collaborative problem solving and iterative modeling optimization based on the forward thought construction and backward reflection mechanism Xiao et al. [2024]. Each agent in the framework is initialized with a role that gives context on the type of work or feedback they are to give in the framework. Then each agent is allowed to access an external knowledge base and perform top-k retrieval against it. In addition, the agents have access to chain of thought prompting techniques that allow forward thinking and backward error checking in the reasoning process. If the output is too long through the reasoning process, the agent can choose to summarize it's output. Lastly, the agent can comment on the work that it has produced, in which the agent can give comments or suggestions Xiao et al. [2024].

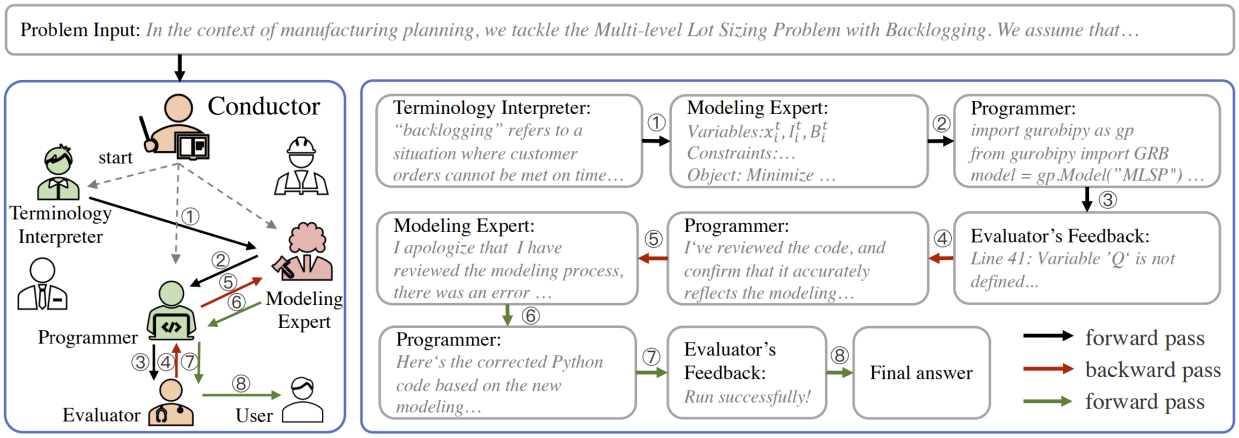

Figure 1: An example illustration of the workflow of Chain-of-Experts Xiao et al. [2024]

.

Orchestrating the experts through the conductor expert contributes to the success the model has had with the LPWP dataset.

Optimius AhmadiTeshnizi et al. [2025] is a similar framework as Chain of Experts where multiple experts are orchestrated in order to solve the word problem using a LP formulation. The main difference between Optimus and CoE is Optimus maints a connection graph (tripartite) to keep track of variables, parameters, constraints, and the objective. This connection graph allows the model to select certain parts of the graph when giving feedback between experts.

4 Experiments

To explore the effectiveness of the conductor in the multi-agent framework in solving operation research problems. Our approach will use different models for the conductor to evaluate their effectiveness in conducting the experts. Rather than just using an LLM, the framework can use an RL agent, a simple rule based model or a smaller LLM to navigate the between the experts. Is there a good model to pick which expert to consult in debugging the model? For the RL agent the aim is to explore different rewards and reinforcement learning algorithms to find the optimal interaction between the experts when developing the IP model. Whereas the simple ruled based algorithm will choose the models based on static domain knowledge to efficiently choose applicable experts while making use of dynamic complexity determination.

4.1 Dataset

The LPWP dataset includes examples from the Nl4OPT dataset and have been further annotated Ramamonjison et al [2022]. Each entry contains a natural language description of a problem that lays out the objective and constraints that a model must optimize and satisfy. These entries include a ground truth model in the LP format that gives the correct solution to the problem. The authors of Nl4OPT recommend creating an intermediate representation that can then be used to create the model in a LP format. The standard metric used for these datasets is to achieve a good accuracy on the correctness of solutions while simultaneously decreasing the compile and runtime errors.

4.2 Reinforcement Learning Policy

[1] Creating a conductor that learns the next expert to choose in the LP formulation process can be expressed in a reinforcement learning framework. Here, a model learns an efficient policy to choose experts until the final produced code is correct. The policy used to evaluate the current status depends

[1] RL Policy Github

on the correctness of the code.

$$\begin{cases} 1 \in \text{Code is correct} \\ 0 \in \text{otherwise} \end{cases}$$

ground truth code is provided and the goal of the experts is to deliver an LP formulation that matches the ground truth. The hamming distance is taken between the ground truth formulation and the experts generated formulation. The action space describes what the model is allowed to choose in the environment. In the framework, the conductor is allowed to freely choose any of the current experts that are available. The experts include the Terminology Interpreter, Parameter Extractor, Modeling Expert, Programming Example Provider, Programming Expert, Modeling Knowledge Supplement Expert, and Code Reviewer. Then the state space corresponds to the current expert in use and the comment from that expert. The comment by the current expert is either the generated code, the intermediate representation of the LP, or just text about the current lp model formulation. Following the algorithm from Volodymyr et. al, called Deep Q-learning, we created an environment that fits the action space, state space, and the policy for a deep q-network to learn in Mnih et al. [2015]. The Q network consists of 2 dense layers of size 100 and 50, both with RELU as activation functions. Then a final dense layer used to predict the q-value based on the action/expert chosen. This model is trained for 200,000 iterations in this environment, and the best resulting policy is used as the conductor.

4.3 Rule-Based

[2] In a multi-expert collaborative system, the efficacy of the decision-making process greatly depends on selecting and sequencing experts appropriately according to their specialization and problem requirements. To address this, we propose a static, expert selection, and ordering approach that leverages keyword-based heuristic scoring combined with targeted complexity assessment via an LLM. This approach operates in two distinct phases:

1. Suitability Scoring:

   Initially, experts are selected based on keyword-weighted scoring of the provided problem description. Each expert is associated with a unique set of domain-specific keywords, each assigned a predefined weight reflecting its relevance to the expert's area of expertise. We compute a suitability score for each candidate expert by summing the weights of keyword occurrences within the problem context. Experts with the highest aggregate scores are thus identified as the most suitable for initial consultations.

   Formally, we define the suitability score S_e for expert e as:

   $$S(e) = \sum_{k \in K_e} w_k \cdot f(k, D)$$

   where:

   - K_e represents the set of keywords associated with expert e
   - w_k denotes the weight assigned to keyword k
   - $f(k, D)$ is a binary indicator function evaluating to 1 if keyword appears in the problem description D, and 0 otherwise.

   Experts whose scores fall below a minimum predefined threshold are typically excluded from selection, ensuring that consultations remain targeted and relevant.

2. Complexity Assessment:

   Although keyword-based scoring effectively determines general expert suitability, certain complex problems require additional oversight, particularly in programming and subsequent code verification. To address this, we integrated an LLM expert defined to the task of determining the necessity of performing a dedicated code review by predicting the complexity of the implementation and the likelihood of errors. When a code review

[2] Rule-Based Github

is deemed necessary, domain experts are consulted first according to suitability score until the collaborator limit reaches two, the mandated Programming Expert is consulted, followed by the Code Reviewer as the last consultation.

4.4 Different LLM Models for Experts

We explored using alternative LLM APIs as a replacement for the standard GPT models to further decrease operational costs. By evaluating a lower-cost LLM option, we aimed to determine if a less expensive model could achieve comparable performance to GPT-based conductors and experts. The goal was to understand the trade-off between model performance and cost efficiency. Though in testing GPT-4o-mini and Gemini-2.0-flash-lite, the performance was too low to determine an accurate comparison.

5 Results

Method	Accuracy %	CE Rate %	RE Rate %
COE w/o Experts w/o Conductor	40.75	35.07	2.78
COE w/o Conductor	30.56	28.47	24.31
COE	**46.88**	**2.43**	23.26
COE w/ RL Policy	31.98	38.89	13.86
COE w/ Rule-Based Policy	44.90	9.68	**22.54**

Table 1: Accuracy, compile error rate, and runtime error rate of COE based on model. All tested on the LPWP dataset, with GPT-3.5-Turbo, iterations set to 3, with each iteration consisting of 3 forward steps.

Model	Tokens (thousands)
COE w/o Conductor	4,668
COE	4,248
COE w/ RL Policy	4,512
COE w/ Rule-Based Policy	**1,732**

Table 2: Token cost of COE for the results shown in Table 1.

6 Discussion

Our experimental evaluation compared various versions of the Chain of Experts (CoE) framework based on accuracy, compile-error (CE) rate, runtime-error (RE) rate, and token usage. Among the tested policies, the reinforcement learning (RL) approach served primarily as a theoretical reference. Due to computational resource constraints and high training costs, extensive training of the RL policy was not feasible, resulting in performance similar to random expert selection. This underscores the practical limitations of implementing RL policies without sufficient training resources.

On the other hand, the rule-based policy demonstrated strong performance, achieving results comparable to the original LLM-based conductor. Importantly, the rule-based approach significantly improved efficiency, consuming approximately 60% fewer tokens compared to the traditional conductor-based CoE. This indicates that a simple yet carefully designed rule-based conductor can substantially reduce computational costs without a notable sacrifice in performance.

Expert	Count
Paramater Extractor	288
Programming Expert	288
Code Reviewer	106
Modeling Expert	124
Terminology Interpreter	43
Modeling Knowledge Supplement Expert	15

Table 3: Count of experts selected for the Rule-Based Policy

7 Conclusion

This study highlights that while sophisticated approaches like reinforcement learning may offer theoretical promise, practical constraints often limit their feasibility. Though they may show glimmers of practicality if training adequately allows for the policy to extend to a wider net of problems effectively. Conversely, straightforward and well-constructed rule-based methods offer a compelling alternative that can maintain high performance at a substantially lower computational expense. Future research may further refine these simple policies, further bridging the gap between theoretical complexity and practical applicability in real-world operational optimization problems.

References

Ali AhmadiTeshnizi, Wenzhi Gao, Herman Brunborg, Shayan Talaei, Connor Lawless, and Madeleine Udell. Optimus-0.3: Using large language models to model and solve optimization problems at scale, 2025. URL https://arxiv.org/abs/2407.19633.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin A. Riedmiller, Andreas Kirkeby Fidjeland, Georg Ostrovski, Stig Petersen, Charlie Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015. URL https://api.semanticscholar.org/CorpusID:205242740.

. Ramamonjison et al. Augmenting operations research with auto-formulation of optimization models from problem descriptions. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 29–62, Abu Dhabi, UAE, December 2022. Association for Computational Linguistics. URL https://aclanthology.org/2022.emnlp-industry.4.

Ziyang Xiao, Dongxiang Zhang, Yangjun Wu, Lilin Xu, Yuan Jessica Wang, Xiongwei Han, Xiaojin Fu, Tao Zhong, Jia Zeng, Mingli Song, and Gang Chen. Chain-of-experts: When LLMs meet complex operations research problems. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=HobyL1B9CZ.